Rideshare. Review the following Migration file that added the constraint to the database.

ruby/migration_trip_rating_check_constraint.rb

```
class AddTripRatingCheckConstraint < ActiveRecord::Migration[7.0]
  def change
   add_check_constraint :users,
     "rating >= 1 AND rating <= 5",
     name: "rating_check"
  end
end
```

The Migration uses add_check_constraint (added in Rails 7) for the trips table and rating column. The constraint boolean logic is very similar to what's in the Active Record Ruby code although it's written in SQL.

Besides boolean logic, the database column itself acts as a form of validation since it only allows compatible values for the type. For example, a floating point value cannot be stored in the rating column with an integer type.

Strong Migrations flags adding a CHECK constraint and validating it in one operation.

As a safer variation, it suggests adding the constraint without validating it. You'll look at how to do that next.

Review these Rideshare migrations:

sh/migration_check_constraint_dangerous.rb

```
# Example message from Strong Migrations:
#
# === Dangerous operation detected #strong_migrations ===
#
# Adding a check constraint key blocks
# reads and writes while every row is checked.
# Instead, add the check constraint without
# validating existing rows, then validate them
# in a separate migration.

# Add a migration with "validate: false"
class AddTripRatingCheckConstraint < ActiveRecord::Migration[7.0]
  def change
    add_check_constraint :trips,
    "rating >= 1 AND rating <= 5",
    name: "rating_check",
    validate: false
  end
end

# Second migration, validates the constraint
class ValidateAddTripRatingCheckConstraint < ActiveRecord::Migration[7.0]
```